

Pseudo-code Guide

Learning points:

- ☐ Pseudo-code is a form of structured English meant to convey the meaning of a program, normally to laymen in easily and commonly understood format
 - ☐ Understand the syntax and Usage of Pseudo code GCE A level standard
-

Keywords

Data Types Constants and Variables

Data Types

- `INTEGER` a whole number
- `REAL` a number capable of containing a fractional part
- `CHAR` a single character
- `STRING` a sequence of zero or more characters
- `BOOLEAN` the logical values TRUE and FALSE
- `DATE` a valid calendar date

Variable

Variable declaration

```
DECLARE <identifier> : <data type>
```

Variable assignment

```
<identifier> <- <someData>
```

- can be used to reassign a variable - `index <- index + 1`

Constant Declaration

```
CONSTANT <identifier> = <value>
```

- for when defining a constant identifier is more meaningful than using a literal
- normally defined at the start of the scope or the code

Arrays

Arrays are a fixed size structure requiring same data type for all its elements

Declaring `ARRAY` - requires a lower bound, upper bound and a data type

```
// 1 dimensional array  
DECLARE <identifier>:ARRAY[<lowerBound>:<upperBound>] OF <data type>
```

```
// 2 dimensional array  
DECLARE <identifier>:ARRAY[<lowerBound>:<upperBound>] OF <data type>
```

Accessing Arrays

access - `<identifier>[<index>]`

assign - `<identifier>[<index>] <- <newValue>`

Logic

IF / THEN / ELSE / IF ELSE / ENDIF

```
//one condition  
IF <condition> THEN  
    ...  
ENDIF  
  
//one condition and else  
IF <condition> THEN  
    ...  
ELSE  
    ...  
ENDIF  
  
//multiconditional  
IF <condition1>  
    ...  
ELSE IF <condition2>  
    ...  
...  
ELSE  
    ...  
ENDIF
```

Loops

 [Iterator >](#)



FOR - takes some integer `<lower>` and `<upper>`, iterates from provided `<lower>` to `<upper>`

```
FOR <identifier> <- <lower> TO <upper>
  ...
ENDFOR
```

* includes `TO` and `ENDFOR` keywords

** note that identifier may be accessed as current iterated number

WHILE - pre-conditional loop, only iterates if condition checked at the start resolves to `TRUE`

```
WHILE <condition>
  ...
ENDWHILE
```

* includes `ENDWHILE` keyword

REPEAT - post-conditional loop, only iterates if condition checked at the end resolves to `TRUE`

```
REPEAT
    ...
UNTIL <condition>
```

Operations

Pseudo-code operations have certain functionality and usage to be followed

INPUT / **OUTPUT** - writes to and reads from an identifier respectively

```
INPUT <identifier>

OUTPUT <identifier>
OUTPUT "sometext", <identifier>, "sometext"
```

* **OUTPUT** may receive multiple items, delimited by **,**

+ / **-** / ***** / **/** - standard arithmetic operations

DIV / **MOD** - finds quotient and remainder of a number after division

```
<identifier> DIV <number>
<identifier> MOD <number>
```

> / **<** / **=** - standard comparison operator, returns a **BOOLEAN**

<= / **>=** / **<>** - additional comparison operators, less than equals, greater than equals and non equal respectively, returns a **BOOLEAN**

AND / **OR** / **NOT** - takes in two **BOOLEAN**s respectively on left and right, and returns a **BOOLEAN**

true conditions:

- **AND** - both conditions true
- **OR** - one conditions true
- **NOT** - conditions do not match, similar to **<>**

```
IF <variableA> = <someNumber> AND <variableB> = <someOtherNumber>
    THEN
        ...
ENDIF

IF <variableA> = <someNumber> OR <variableB> = <someOtherNumber>
    THEN
        ...
ENDIF
```

```
IF <variableA> = <someNumber> NOT <variableB> = <someOtherNumber>
  THEN
    ...
ENDIF
```

RIGHT(s, n), **LEFT(s, n)** - takes in a **STRING** **s** and length **n** of sub-string to retrieve from right and left respectively

```
OUTPUT RIGHT('abcdef', 3)
> def

OUTPUT LEFT('abcdef', 3)
> abc
```

MID(s, l, n) - takes in a **STRING** **s**, lower bound **l** and length **n** of sub-string to retrieve

```
OUTPUT MID('abcdef', 2, 3)
> bcd
```

LCASE(s), **UCASE(s)** - takes a **STRING** **s**, returns all lower or uppercase respectively

```
OUTPUT LCASE('HELLOWORLD!')
> helloworld!

OUTPUT UCASE('helloworld!')
> HELLOWORLD!
```

& - concatenates two strings on the left and right together

```
<identifier> <- "summer" & " " & "pudding"
output <identifier>
> summer pudding
```

INT(x) - takes a **REAL** **x** and returns a **INTEGER** containing the whole number portion of **x**

RAND(x) - takes an **INTEGER** **x** and returns a random **REAL** from 0 to **x**

Procedures and Function

PROCEDURE - a reusable block of code which does not return anything

```
PROCEDURE <identifier>(<param1>: <data type1>, <param2>: <data type2>,  
...)  
...  
ENDPROCEDURE
```

* Includes additional keyword `ENDPROCEDURE`

PROCEDURE - a reusable block of code which returns a specified data type

```
FUNCTION <identifier>(<param1>: <data type1>, <param2>: <data type2>, ...)  
RETURNS <data type>  
...  
ENDFUNCTION
```

* Includes additional keywords `ENDFUNCTION`, `RETURNS` and `RETURN`